# 피어 발견과 연결 관리

## 개요

CometBFT 네트워크에서 피어 발견과 연결 관리는 분산 네트워크의 핵심 기능입니다. 새로운 노드가 네트워크에 참여하거나 기존 노드가 새로운 피어를 찾아 연결하는 과정은 네트워크의 건강성과 확장성을 결정하는 중요한 요소입니다.

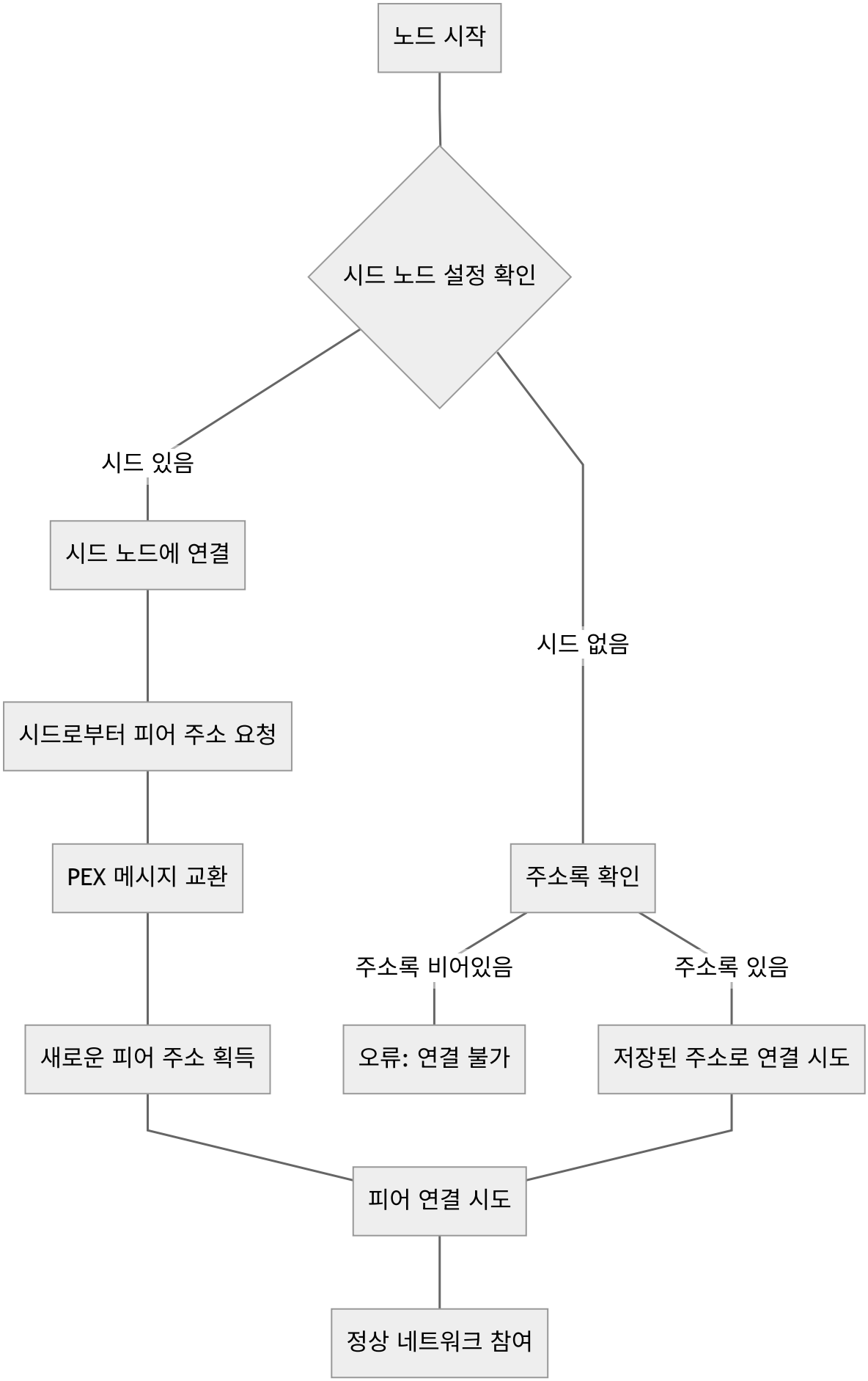
이 문서에서는 다음 내용을 다룹니다:

- 초기 네트워크 참여 과정
- PEX (Peer Exchange) 프로토콜
- 주소록(AddrBook) 관리
- 연결 생명주기 관리
- 피어 품질 평가 시스템

## 초기 네트워크 참여 과정

### 1. 부트스트랩 과정

새로운 노드가 CometBFT 네트워크에 처음 참여할 때의 과정입니다:




## 2. 시드 노드 (Seed Nodes)

시드 노드는 새로운 노드의 네트워크 참여를 돕는 특별한 역할을 합니다.

**시드 노드의 특징**

- **부트스트랩 지원**: 새로운 노드에게 초기 피어 목록 제공
- **일시적 연결**: 피어 주소 제공 후 연결 종료
- **높은 가용성**: 항상 접근 가능한 안정적인 노드
- **주소 수집**: 다양한 피어 주소를 수집하고 관리

**시드 노드 설정**

```
# config.toml

[p2p]

seeds = "node1@192.168.1.100:26656,node2@192.168.1.101:26656"

seed_mode = true # 이 노드를 시드 모드로 실행
```

## 3. 지속적 피어 (Persistent Peers)

지속적 피어는 항상 연결을 유지하려고 시도하는 특별한 피어입니다.

**특징**

- **자동 재연결**: 연결이 끊어지면 자동으로 재연결 시도
- **우선 연결**: 다른 피어보다 우선적으로 연결 유지
- **네트워크 안정성**: 네트워크 분할 상황에서 연결성 보장

**설정 예시**

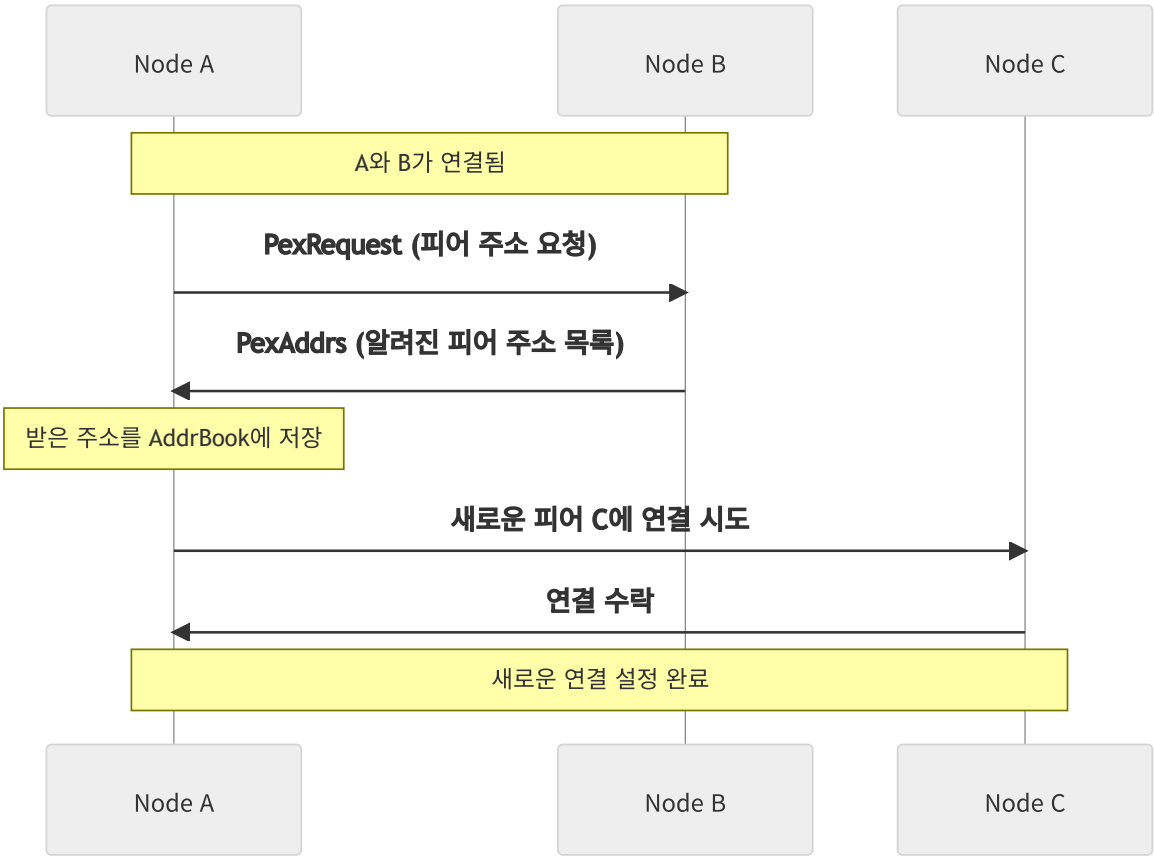
```
# config.toml
```

```
[p2p]

persistent_peers = "validator1@10.0.0.1:26656,validator2@10.0.0.2:26656"
```

# PEX (Peer Exchange) 프로토콜

PEX는 피어 간에 알려진 다른 피어들의 주소를 교환하는 프로토콜입니다.

## 1. PEX 프로토콜 동작 원리




## 2. PEX 메시지 유형

### PexRequest

```
message PexRequest {}
```

- 다른 피어에게 알려진 피어 주소를 요청하는 메시지
- 빈 메시지로 단순히 요청 의도만 전달

**PexAddrs**

```
message PexAddrs {
repeated NetAddr addrs = 1;
}
```

- 알려진 피어 주소 목록을 응답하는 메시지
- 최대 16개의 주소를 포함

## 3. PEX Reactor 구현

```
type Reactor struct {
book AddrBook
requestsSent *cmap.CMap // 요청 추적
lastReceivedRequests *cmap.CMap // 요청 빈도 제한
}

func (r *Reactor) RequestAddrs(p Peer) {
id := p.ID()
if  r.requestsSent.Has(id) {
return  // 이미 요청 중
```

```
}

r.requestsSent.Set(id, struct{}{})

p.Send(p2p.Envelope{

ChannelID: PexChannel,

Message: &tmp2p.PexRequest{},

})

}
```

### 4. PEX 보안 메커니즘

**요청 빈도 제한**

- **최소 간격**: 피어당 최소 10초 간격으로 요청 제한
- **중복 방지**: 동일 피어에 대한 중복 요청 차단
- **DoS 방어**: 과도한 요청으로부터 보호

**응답 크기 제한**

- **최대 주소 수**: 한 번에 최대 16개 주소만 전송
- **메시지 크기**: 전체 메시지 크기 제한
- **선택적 응답**: 품질 좋은 피어 주소 우선 제공

## 주소록 (AddrBook) 관리

AddrBook은 알려진 피어들의 주소와 상태 정보를 관리하는 핵심 컴포넌트입니다.

### 1. 주소록 구조

```go
type addrBook struct {
ourAddrs  map[string]struct{} // 자신의 주소들
privateIDs  map[nodekey.ID]struct{} // 비공개 피어 ID들
addrLookup  map[nodekey.ID]*knownAddress // 주소 검색 테이블
badPeers  map[nodekey.ID]*knownAddress // 차단된 피어들
bucketsOld []map[string]*knownAddress // 검증된 주소 버킷
bucketsNew []map[string]*knownAddress // 새로운 주소 버킷
}
```

## 2. 버킷 시스템

AddrBook은 피어 주소를 두 가지 유형의 버킷으로 분류합니다:

**New Buckets (새로운 주소 버킷)**

- **미검증 주소**: 아직 연결해보지 않은 주소들
- **1024개 버킷**: 각 버킷당 최대 64개 주소
- **확률적 추가**: 중복 주소는 확률적으로만 추가

**Old Buckets (검증된 주소 버킷)**

- **검증된 주소**: 성공적으로 연결했던 주소들
- **256개 버킷**: 각 버킷당 최대 64개 주소
- **높은 신뢰도**: 연결 시 우선적으로 선택

## 3. 주소 선택 알고리즘

```go
func (a *addrBook) PickAddress(biasTowardsNewAddrs  int) *na.NetAddr {
// biasTowardsNewAddrs: 0-100, 새 주소 선택 편향도
oldCorrelation := math.Sqrt(float64(a.nOld)) * (100.0 - float64(biasTowardsNewAd
newCorrelation := math.Sqrt(float64(a.nNew)) * float64(biasTowardsNewAddrs)
pickFromOldBucket := (newCorrelation+oldCorrelation)*a.rand.Float64() < oldCorre
// 선택된 버킷 유형에서 랜덤하게 주소 선택
if  pickFromOldBucket {
return  a.pickFromOldBucket()
} else {
return  a.pickFromNewBucket()
}
}
```

## 4. 주소 품질 관리

### 주소 상태 추적

```go
type knownAddress struct {
Addr *na.NetAddr
Src *na.NetAddr // 주소를 알려준 소스
Attempts  int  // 연결 시도 횟수
LastAttempt time.Time // 마지막 시도 시간
LastSuccess time.Time // 마지막 성공 시간
```

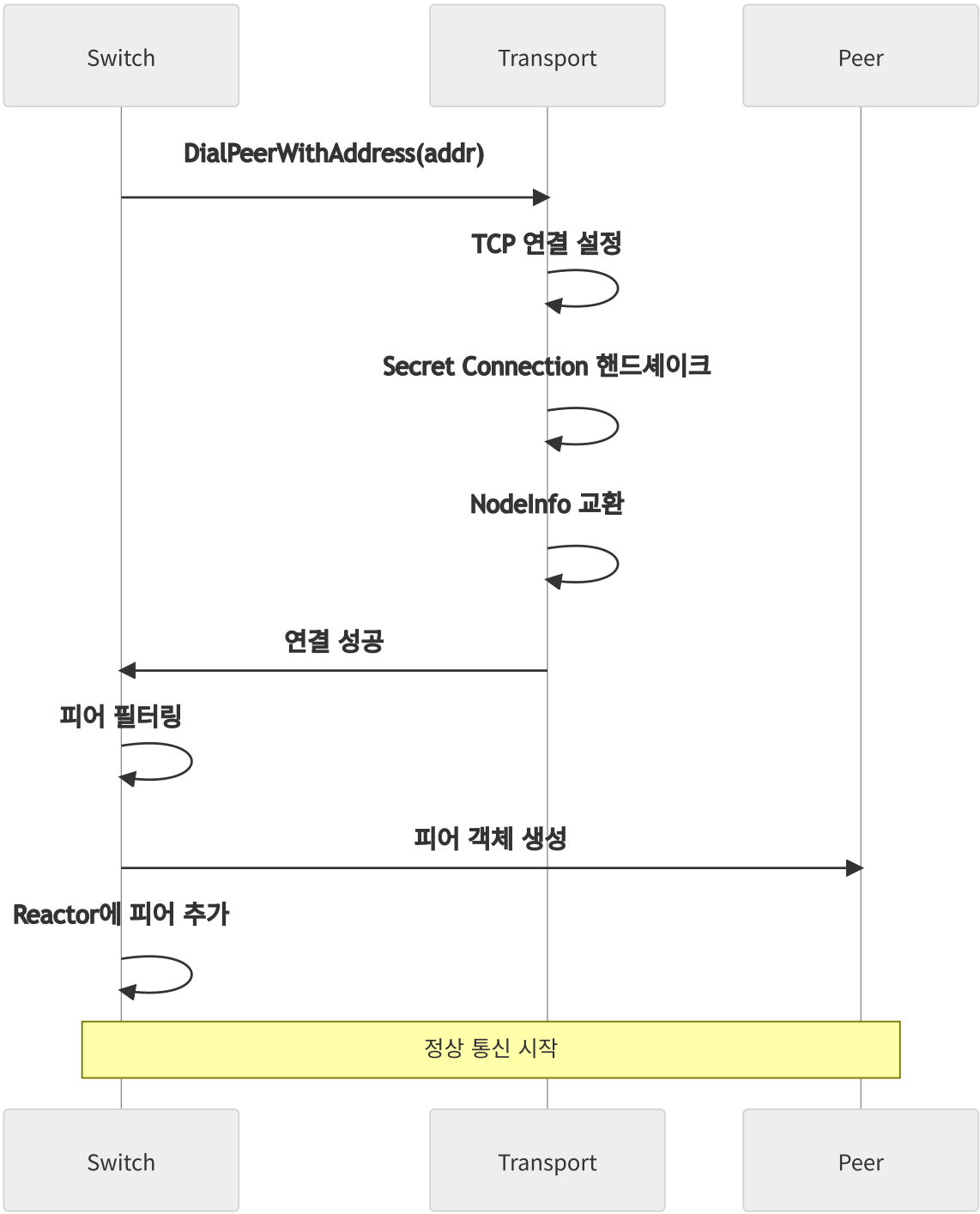
```
BucketType  byte  // 버킷 유형 (new/old)

Buckets []int  // 속한 버킷들

}
```

**품질 평가 기준**

- **연결 성공률**: 시도 대비 성공 비율
- **최근 활동**: 최근 연결 성공 시간
- **응답 시간**: 연결 설정 소요 시간
- **안정성**: 연결 유지 시간

## 연결 생명주기 관리

### 1. 연결 설정 과정




## 2. 연결 상태 모니터링

### 연결 품질 메트릭

- **지연 시간**: 메시지 왕복 시간
- **처리량**: 초당 전송 바이트 수

- **오류율**: 전송 실패 비율
- **큐 상태**: 송신 큐 사용률

### 상태 확인 메커니즘

```
func (p *peer) eventLoop() {
metricsTicker := time.NewTicker(metricsTickerDuration)
defer  metricsTicker.Stop()
for {
select {
case  err := <-p.Conn.ErrorCh():
p.onPeerError(p, err)
return
case <-metricsTicker.C:
p.updateMetrics()
case <-p.Quit():
return
}
}
}
```

## 3. 재연결 메커니즘

### 지수 백오프 재연결

```go
func (sw *Switch) reconnectToPeer(addr *na.NetAddr) {
// 고정 간격 재시도 (약 2분)
for i := 0; i < reconnectAttempts; i++ {
err := sw.DialPeerWithAddress(addr)
if err == nil {
return // 성공
}
time.Sleep(reconnectInterval)
}
// 지수 백오프 재시도 (최대 ~24시간)
for i := 1; i <= reconnectBackOffAttempts; i++ {
sleepDuration := time.Duration(math.Pow(3, float64(i))) * time.Second
time.Sleep(sleepDuration)
err := sw.DialPeerWithAddress(addr)
if err == nil {
return // 성공
}
}
}
```

## 피어 품질 평가 시스템

### 1. 피어 분류

**Good Peers (좋은 피어)**

- **성공적 연결**: 최근에 성공적으로 연결된 피어
- **안정적 통신**: 오류 없이 안정적으로 통신하는 피어
- **Old Bucket 저장**: 검증된 주소 버킷에 저장

**Bad Peers (나쁜 피어)**

- **연결 실패**: 반복적으로 연결에 실패하는 피어
- **악의적 행동**: 잘못된 데이터를 전송하는 피어
- **일시적 차단**: 설정된 시간 동안 연결 차단

## 2. 평판 관리

**점수 기반 시스템**

```
func (a *addrBook) MarkGood(id nodekey.ID) {
ka := a.addrLookup[id]
if ka == nil {
return
}
ka.markGood()
if ka.isNew() {
a.moveToOld(ka) // New에서 Old 버킷으로 이동
}
}
```

```
func (a *addrBook) MarkBad(addr *na.NetAddr, banTime time.Duration) {
if  a.addBadPeer(addr, banTime) {
a.removeAddress(addr) // 주소록에서 제거
}
}
```

### 자동 복구 메커니즘

```
func (a *addrBook) ReinstateBadPeers() {
for  _, ka := range  a.badPeers {
if  ka.isBanned() {
continue  // 아직 차단 기간 중
}
// 새로운 버킷에 다시 추가
bucket := a.calcNewBucket(ka.Addr, ka.Src)
a.addToNewBucket(ka, bucket)
delete(a.badPeers, ka.ID())
}
}
```

## 네트워크 최적화 전략

### 1. 연결 다양성

#### 지리적 분산

- **다양한 지역**: 여러 지역의 피어와 연결
- **네트워크 다양성**: 다른 ISP의 피어 선호
- **지연 최적화**: 지리적으로 가까운 피어 우선

### IP 그룹 분산

```
func (a *addrBook) groupKey(na *na.NetAddr) string {
if  ipv4 := na.IP.To4();  ipv4 != nil {
return  na.IP.Mask(net.CIDRMask(16, 32)).String() // /16 서브넷
}
// IPv6의 경우 /32 또는 /36 서브넷 사용
return  na.IP.Mask(net.CIDRMask(32, 128)).String()
}
```

## 2. 적응적 연결 관리

### 동적 피어 수 조절

- **네트워크 상황**: 네트워크 상태에 따른 연결 수 조절
- **리소스 사용량**: CPU/메모리 사용량 고려
- **대역폭 관리**: 사용 가능한 대역폭에 따른 조절

### 지능적 피어 선택

```
func (r *Reactor) ensurePeers() {
out, in, dial := r.Switch.NumPeers()
numToDial := r.Switch.MaxNumOutboundPeers() - (out + dial)
```

```
// 연결 수가 적을 때는 검증된 피어 선호
newBias := cmtmath.MinInt(out, 8)*10 + 10
for i := 0; i < numToDial; i++ {
addr := r.book.PickAddress(newBias)
if addr != nil {
go r.dialPeer(addr)
}
}
}
```

# 문제 해결 및 디버깅

## 1. 일반적인 연결 문제

### 시드 노드 연결 실패

```
# 로그 확인
grep "Error dialing seed" ~/.cometbft/logs/cometbft.log

# 네트워크 연결 테스트
telnet <seed_ip> 26656
```

### 피어 발견 실패

```
# 주소록 상태 확인
curl http://localhost:26657/net_info

# PEX 상태 확인
grep "PEX" ~/.cometbft/logs/cometbft.log
```

## 2. 성능 모니터링

### 연결 메트릭

- `cometbft_p2p_peers` : 현재 연결된 피어 수
- `cometbft_p2p_peer_receive_bytes_total` : 수신 바이트 수
- `cometbft_p2p_peer_send_bytes_total` : 송신 바이트 수

### 주소록 메트릭

- 총 주소 수
- New/Old 버킷 분포
- Bad 피어 수

## 3. 설정 최적화

### 연결 관련 설정

```
[p2p]
max_num_inbound_peers = 40
max_num_outbound_peers = 10
```

```
flush_throttle_timeout = "100ms"

max_packet_msg_payload_size = 1024

send_rate = 5120000 # 5MB/s

recv_rate = 5120000 # 5MB/s
```

**PEX 관련 설정**

```
[p2p]

pex = true

addr_book_strict = true

max_num_inbound_peers = 40

max_num_outbound_peers = 10
```

이러한 피어 발견과 연결 관리 메커니즘을 통해 CometBFT는 자율적이고 확장 가능한 P2P 네트워크를 구축하며, 새로운 노드의 참여와 기존 노드 간의 효율적인 연결을 보장합니다.